



Universidad Nacional de La Matanza

Caravana del Desierto

Algoritmos y Estructura de datos (3640)

Docentes:

- Guatelli, Renata
- Witold Martínez, Pablo

Alumnos:

- Apollaro, Dasha 44448125
- Salguero, Candela 44759623
- Torres Moran, María Celeste 44005719
- Villar, Facundo Agustin 45415804

Cuatrimestre: 1º cuatrimestre 2026

Año lectivo: 2026

Manual del Proyecto

Caravana del Desierto

1. Introducción

Caravana del Desierto es un juego desarrollado en lenguaje **C**, en el cual una caravana controlada por el jugador debe atravesar una ruta desértica desde el **Campamento Inicial**, representado con el símbolo **I**, hasta la **Ciudad Refugio**, representada con el símbolo **S**.

Durante el recorrido, el jugador deberá avanzar o retroceder según el resultado de un dado virtual, evitando obstáculos como **bandidos** y **tormentas de arena**, y aprovechando recursos como **premios**, **vidas extra** y **oasis**.

El objetivo principal del juego es llegar a la Ciudad Refugio antes de perder todas las vidas disponibles.

Desde el punto de vista de la implementación, la ruta del desierto se modela mediante una **lista circular doblemente enlazada**, mientras que la administración de jugadores utiliza una capa de datos indexada mediante un **árbol de búsqueda binaria**.

2. Objetivo del juego

El objetivo del jugador es desplazarse por la ruta del desierto desde la posición inicial **I** hasta la salida **S**.

La partida finaliza en alguno de los siguientes casos:

- El jugador llega a la Ciudad Refugio **S**.
- El jugador pierde todas sus vidas antes de alcanzar la salida.

Durante la partida, el jugador acumula puntos al capturar premios y puede aumentar sus posibilidades de supervivencia obteniendo vidas extra u oasis.

3. Reglas principales del juego

En cada turno, el jugador arroja un dado virtual que genera un número aleatorio entero entre **1 y 6**.

Ese valor indica exactamente la cantidad de posiciones que el jugador debe desplazarse. El jugador puede decidir si moverse hacia adelante o hacia atrás sobre la ruta.

El movimiento se resuelve solamente en la posición final. Esto significa que las posiciones intermedias no producen efectos sobre el jugador.

Por ejemplo, si el dado indica 4, el jugador debe desplazarse exactamente 4 posiciones. Solo se evalúa lo que ocurre en la casilla donde termina el movimiento.

4. Elementos del juego

Símbolo	Elemento	Descripción
I	Campamento Inicial	Posición desde la cual comienza el jugador
S	Ciudad Refugio	Meta final del recorrido
J	Jugador	Caravana controlada por el usuario
B	Bandido	Enemigo que puede interceptar al jugador
P	Premio	Otorga 1 punto al jugador
V	Vida extra	Incrementa en 1 las vidas disponibles
O	Oasis	Protege al jugador durante el siguiente turno
T	Tormenta	Hace perder el siguiente turno, salvo que el jugador esté protegido

5. Funcionamiento de los turnos

Cada turno se divide en distintas etapas:

1. Se muestra el estado actual de la ruta.
2. El jugador arroja el dado virtual.
3. El jugador decide si avanza o retrocede.
4. Se calcula la posición de destino.
5. Se aplica el efecto de la casilla final.
6. Se mueven los bandidos.
7. Se verifica si algún bandido interceptó al jugador.
8. Se registra el movimiento realizado.
9. Se evalúa si la partida finalizó.

6. Movimiento del jugador

El jugador se mueve según el valor obtenido en el dado.

Si el jugador se desplaza hacia adelante y sobrepasa la Ciudad Refugio S, no se queda fuera del tablero. En ese caso, debe retroceder la cantidad de posiciones sobrantes, cumpliendo siempre con el movimiento exacto indicado por el dado.

Por ejemplo:

Si la salida está en la posición 20, el jugador está en la posición 18 y el dado indica 5:

18 -> 19 -> 20 -> 19 -> 18 -> 17

El jugador termina en la posición 17, porque al llegar a la salida y sobrar movimientos, rebota hacia atrás.

Esta fue una decisión importante del diseño, ya que respeta la regla de que el jugador debe moverse exactamente la cantidad indicada por el dado.

7. Movimiento de los bandidos

Los bandidos se desplazan automáticamente durante el turno de la computadora.

A diferencia del jugador, los bandidos se mueven de forma circular, sin rebote. Esto significa que pueden cruzar tanto el inicio como la salida como si la ruta fuera un círculo.

El movimiento de los bandidos se calcula utilizando la cantidad total de posiciones de la ruta.

Si un bandido termina en la misma posición que el jugador, el jugador pierde una vida, vuelve al Campamento Inicial y el bandido que lo interceptó es eliminado de la ruta.

8. Efectos de las posiciones

Premios P

Cuando el jugador cae en una posición con premio, obtiene **1 punto**.

Vidas extra V

Cuando el jugador cae en una posición con vida extra, incrementa en **1** su cantidad de vidas disponibles.

Oasis O

Cuando el jugador llega a un oasis, obtiene protección para el siguiente turno.

Mientras esté protegido, no puede perder vidas por tormentas ni por bandidos.

Tormentas T

Cuando el jugador cae en una tormenta, pierde el siguiente turno.

Si el jugador se encuentra protegido por un oasis, la tormenta no produce efecto.

Bandidos B

Un bandido afecta al jugador únicamente si ambos quedan en la misma posición al finalizar un movimiento.

Esto puede ocurrir:

- Luego del movimiento del jugador.
- Luego del movimiento del bandido.

En ese caso, el jugador pierde una vida, vuelve al inicio y el bandido es eliminado.

9. Configuración del juego

El juego utiliza un archivo de configuración llamado:

Config.txt

Este archivo contiene los parámetros principales del tablero y de la partida. Cada parámetro se escribe con el formato:

clave=valor

Parámetros disponibles

Parámetro	Significado
cantidad_posiciones	Cantidad total de casilleros de la ruta
vidas_inicio	Cantidad de vidas iniciales del jugador
maximo_bandidos	Cantidad máxima de bandidos a distribuir
maximo_premios	Cantidad máxima de premios a distribuir
maximo_vidas_extra	Cantidad máxima de vidas extra a distribuir
maximo_oasis	Cantidad máxima de oasis a distribuir
maximo_tormentas	Cantidad máxima de tormentas a distribuir

Este archivo permite modificar el comportamiento general del juego sin necesidad de recompilar el programa.

10. Estructura del proyecto

El proyecto fue desarrollado en **Code::Blocks** y se encuentra organizado en distintos módulos .c y .h.

La estructura general es la siguiente:

Caravana_del_Desierto/

|

|— Archivos/

| |— caravana.txt

| |— config.txt

| |— indiceJugadores.idx

| |— jugadores.dat

| |— partidas.dat

|

|— EstructurasDatos/

| |— TDA_Arbol/

| | |— Arbol_Funciones.c

| | |— Arbol_Header.h

| |

| |— TDA_Cola/

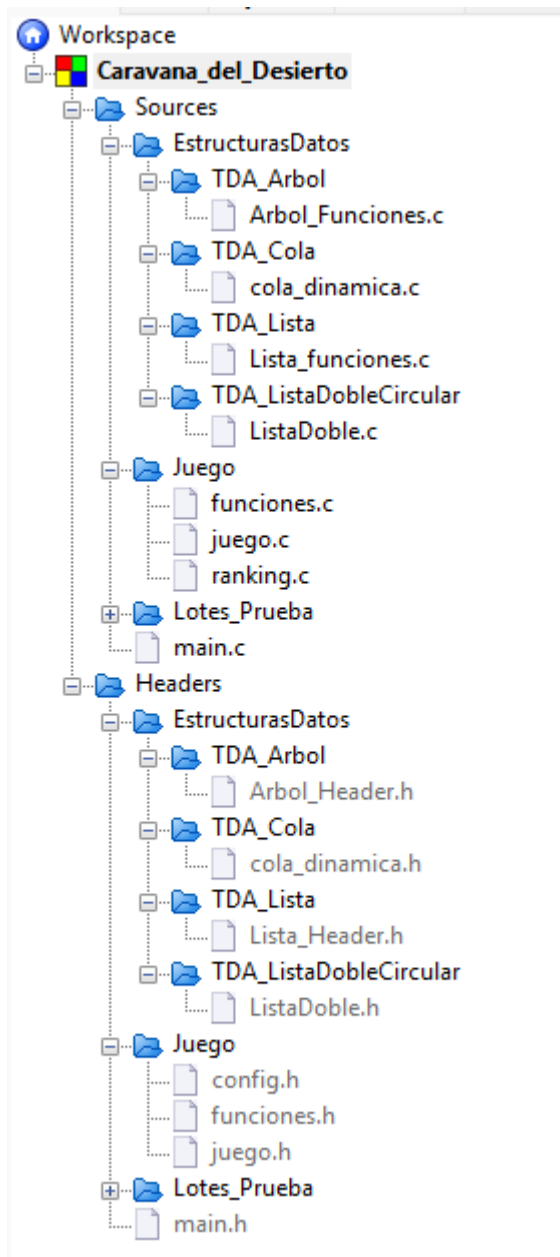
| | |— cola_dinamica.c

| | |— cola_dinamica.h

| |

| |— TDA_Lista/

```
| | └─ Lista_funciones.c
| | └─ Lista_Header.h
| |
| └─ TDA_ListaDobleCircular/
|     └─ ListaDoble.c
|     └─ ListaDoble.h
|
└─ Juego/
    └─ config.h
    └─ funciones.c
    └─ funciones.h
    └─ juego.c
    └─ juego.h
    └─ ranking.c
|
└─ Lotes_Prueba/
|
└─ main.c
└─ main.h
```

11. Descripción de los archivos principales

main.c / main.h

Contiene la entrada principal del programa.

Desde este módulo se inicializan las estructuras necesarias y se ejecuta el menú principal del juego.

Entre sus responsabilidades se encuentran:

- Iniciar el programa.
- Cargar la configuración.
- Ejecutar el menú.

- Llamar a las funciones principales del juego.
- Permitir el alta de jugadores.
- Iniciar partidas.
- Mostrar el ranking.

`funciones.c / funciones.h`

Contiene funciones generales utilizadas por el programa.

Incluye funcionalidades relacionadas con:

- Lectura de la configuración.
- Generación del dado virtual.
- Manejo de errores.
- Alta de jugadores.
- Administración del índice de jugadores.
- Declaración de funciones compartidas, como la muestra del ranking.

Este módulo actúa como una capa auxiliar del sistema.

`juego.c / juego.h`

Contiene el motor principal del juego.

Es uno de los módulos más importantes del proyecto, ya que se encarga de:

- Generar el escenario.
- Crear la ruta del desierto.
- Distribuir premios, vidas extra, oasis, tormentas y bandidos.
- Controlar los turnos.
- Mover al jugador.
- Mover a los bandidos.
- Aplicar los efectos de cada posición.
- Determinar si la partida finalizó.
- Registrar los movimientos realizados.

`ranking.c`

Contiene la lógica para calcular y mostrar el ranking de jugadores.

El ranking se arma a partir de la información almacenada en `partidas.dat`.

Para cada jugador se suman los puntos acumulados en todas sus partidas. Luego, se ordenan de mayor a menor puntaje.

En caso de empate, se utiliza como criterio de desempate la menor cantidad total de movimientos, siendo esta un acumulado de los valores que obtuvo tirando los dados a lo largo de sus partidas.

`lotes_de_prueba.c / lotes_de_prueba.h`

Contiene funciones relacionadas con la carga y persistencia de archivos.

Se utiliza para generar datos de prueba y administrar la escritura y lectura de archivos binarios.

`config.h`

Contiene estructuras, constantes y definiciones compartidas por todo el proyecto.

Este archivo permite centralizar la configuración interna del programa, evitando repetir definiciones en varios módulos.

12. Archivos de datos

`jugadores.dat`

Archivo binario donde se almacenan los datos de los jugadores.

`partidas.dat`

Archivo binario donde se almacenan los datos de las partidas jugadas.

A partir de este archivo se calcula el ranking.

`indiceJugadores.idx`

Archivo binario donde se persiste el índice de jugadores.

Este índice se implementa mediante un árbol de búsqueda binaria.

13. Estructuras de datos utilizadas

13.1 Lista circular doblemente enlazada

La ruta del desierto se representa mediante una **lista circular doblemente enlazada**.

Cada nodo de la lista representa una posición del recorrido.

Esta estructura permite recorrer la ruta tanto hacia adelante como hacia atrás, lo cual es necesario porque:

- El jugador puede avanzar o retroceder.
- Los bandidos pueden moverse en ambos sentidos.
- La ruta debe comportarse como circular para los bandidos.

El TDA utilizado se encuentra en:

`EstructurasDatos/TDA_ListaDobleCircular/`

Archivos:

`ListaDoble.c`

`ListaDoble.h`

Una decisión importante del proyecto fue mantener el encapsulamiento del TDA. Por eso, el módulo `juego.c` no accede directamente a los nodos de la lista, sino que utiliza primitivas del TDA.

13.2 Árbol de búsqueda binaria

Los jugadores se administran mediante una capa de datos indexada por un **árbol de búsqueda binaria**.

Este índice permite realizar búsquedas de jugadores de forma más eficiente.

El índice se guarda en el archivo:

indiceJugadores.idx

El TDA se encuentra en:

EstructurasDatos/TDA_Arbol/

Archivos:

Arbol_Funciones.c

Arbol_Header.h

13.3 Lista simple

La lista simple se utiliza como estructura auxiliar, principalmente para el armado del ranking.

Permite acumular información de jugadores, sumar puntos y movimientos, y luego ordenar los resultados.

El TDA se encuentra en:

EstructurasDatos/TDA_Lista/

Archivos:

Lista_funciones.c

Lista_Header.h

13.4 Cola dinámica

La cola dinámica se utiliza para registrar el historial de movimientos realizados durante la partida.

Se encolan los movimientos del jugador y de los bandidos.

Al finalizar la partida, se muestra el historial con el formato:

FX / BX

Donde:

- F representa el movimiento del jugador hacia adelante.
- B representa el movimiento del jugador hacia atrás.
- X representa la cantidad de posiciones desplazadas.

El TDA se encuentra en:

EstructurasDatos/TDA_Cola/

Archivos:

cola_dinamica.c

cola_dinamica.h

14. Ranking

El ranking se calcula tomando como base todas las partidas jugadas.

Criterio principal

Los jugadores se ordenan por **puntos totales acumulados**, de mayor a menor.

Esto significa que, si un jugador jugó varias partidas, se suman todos los puntos obtenidos en cada una de ellas.

Criterio de desempate

Si dos jugadores tienen la misma cantidad de puntos, queda primero el jugador que haya realizado menos movimientos en total.

Este criterio premia la eficiencia del jugador.

Implementación

La función principal del ranking es:

mostrarRanking

Esta función se encuentra implementada en:

ranking.c

y declarada en:

funciones.h

El ranking lee los datos desde:

partidas.dat

Luego utiliza el TDA Lista para acumular los datos por jugador y ordenar la información antes de mostrarla.

15. Decisiones de diseño del equipo

Durante el desarrollo se tomaron distintas decisiones de diseño para completar aspectos que la consigna dejaba a criterio del grupo.

Movimiento del jugador por número de posición

El movimiento del jugador se resuelve utilizando el número de posición destino, en lugar de recorrer manualmente nodo por nodo desde `juego.c`.

Esto permite mantener más clara la lógica del juego y respetar el encapsulamiento del TDA Lista Circular Doble.

Rebote del jugador al pasar la salida

Si el jugador sobrepasa la Ciudad Refugio, rebota y continúa el movimiento hacia atrás.

Esta decisión cumple con la regla de que el jugador debe moverse exactamente la cantidad indicada por el dado.

Movimiento circular de los bandidos

Los bandidos se mueven de forma circular y sin rebote.

Esto significa que pueden pasar por el inicio y la salida sin detenerse, como si la ruta fuera un círculo.

Encapsulamiento de la lista doble circular

El archivo `juego.c` no accede directamente a los nodos internos de la lista.

No se utilizan accesos como:

- >siguiente
- >anterior
- >info

en el motor del juego.

En su lugar, se utilizan primitivas del TDA, como:

```
buscarPtrElementoListaDC  
desplazarYAplicar  
recorrerListaDC_Condicionada
```

Esto mejora la organización del código y separa correctamente la lógica del juego de la implementación interna de la estructura de datos.

Registro de movimientos

Los movimientos se registran mediante una cola.

Esto permite guardar el historial en el orden en que ocurrieron los eventos y mostrarlo al finalizar la partida.

16. Flujo general de ejecución

El flujo general del programa puede resumirse de la siguiente manera:

Inicio del programa



Carga de configuración



Carga o generación de jugadores



Creación o carga del índice ABB



Menú principal



Alta de jugador / Jugar partida / Ver ranking / Salir



Inicio de partida



Generación de ruta



Distribución de elementos



Turnos del jugador y de los bandidos



Fin de partida



Guardado de datos



Actualización del ranking

17. Menú principal

El programa cuenta con un menú desde el cual el usuario puede acceder a las distintas funcionalidades.

Las opciones principales son:

1. Alta de jugador
2. Jugar partida
3. Mostrar ranking
4. Salir

Desde este menú se controla el funcionamiento general del juego.

18. Finalización de la partida

Una partida puede finalizar por victoria o por derrota.

Victoria

El jugador gana cuando alcanza la Ciudad Refugio S.

Derrota

El jugador pierde si se queda sin vidas antes de llegar a la salida.

Esto puede ocurrir por:

- Ser interceptado por bandidos.
- Caer en tormentas sin protección.
- Perder vidas durante el desarrollo de la partida.

19. Persistencia de datos

El proyecto utiliza archivos binarios para guardar información.

Los datos principales que se persisten son:

- Jugadores.
- Partidas.

- Índice de jugadores.

Esto permite conservar información entre ejecuciones del programa y calcular estadísticas acumuladas, como el ranking.

20. Conclusión

El proyecto **Caravana del Desierto** implementa un juego en lenguaje C aplicando distintas estructuras de datos vistas durante la cursada.

La ruta principal se representa mediante una **lista circular doblemente enlazada**, permitiendo movimientos hacia adelante, hacia atrás y recorridos circulares. Además, se utiliza un **árbol de búsqueda binaria** para indexar jugadores, una **lista simple** para organizar el ranking y una **cola dinámica** para registrar el historial de movimientos.

El desarrollo separa correctamente la lógica del juego, la administración de datos y la implementación de los TDA, favoreciendo la organización, el mantenimiento y la claridad del código.

En conjunto, el proyecto cumple con los requerimientos principales de la consigna y demuestra el uso integrado de múltiples estructuras dinámicas en un programa funcional.